CHAPTER 1

Introduction

1.1 WHY NEURAL NETWORKS AND WHY NOW?

As modern computers become ever more powerful, scientists continue to be challenged to use machines effectively for tasks that are relatively simple for humans. Based on examples, together with some feedback from a "teacher," we learn easily to recognize the letter ***A*** or distinguish a cat from a bird. More experience allows us to refine our responses and improve our performance. Although eventually, we may be able to describe rules by which we can make such decisions, these do not necessarily reflect the actual process we use. Even without a teacher, we can group similar patterns together. Yet another common human activity is trying to achieve a goal that involves maximizing a resource (time with one's family, for example) while satisfying certain constraints (such as the need to earn a living). Each of these types of problems illustrates tasks for which computer solutions may be sought.

Traditional, sequential, logic-based digital computing excels in many areas, but has been less successful for other types of problems. The development of artificial neural networks began approximately 50 years ago, motivated by a desire to try both to understand the brain and to emulate some of its strengths. Early

successes were overshadowed by rapid progress in digital computing. Also, claims made for capabilities of early models of neural networks proved to be exaggerated, casting doubts on the entire field.

Recent renewed interest in neural networks can be attributed to several factors. Training techniques have been developed for the more sophisticated network architectures that are able to overcome the shortcomings of the early, simple neural nets. High-speed digital computers make the simulation of neural processes more feasible. Technology is now available to produce specialized hardware for neural networks. However, at the same time that progress in traditional computing has made the study of neural networks easier, limitations encountered in the inherently sequential nature of traditional computing have motivated some new directions for neural network research. Fresh approaches to parallel computing may benefit from the study of biological neural systems, which are highly parallel. The level of success achieved by traditional computing approaches to many types of problems leaves room for a consideration of alternatives.

Neural nets are of interest to researchers in many areas for different reasons. Electrical engineers find numerous applications in signal processing and control theory. Computer engineers are intrigued by the potential for hardware to implement neural nets efficiently and by applications of neural nets to robotics. Computer scientists find that neural nets show promise for difficult problems in areas such as artificial intelligence and pattern recognition. For applied mathematicians, neural nets are a powerful tool for modeling problems for which the explicit form of the relationships among certain variables is not known.

There are various points of view as to the nature of a neural net. For example, is it a specialized piece of computer hardware (say, a VLSI chip) or a computer program? We shall take the view that neural nets are basically mathematical models of information processing. They provide a method of representing relationships that is quite different from Turing machines or computers with stored programs. As with other numerical methods, the availability of computer resources, either software or hardware, greatly enhances the usefulness of the approach, especially for large problems.

The next section presents a brief description of what we shall mean by a neural network. The characteristics of biological neural networks that serve as the inspiration for artificial neural networks, or *neurocomputing*, are also mentioned. Section 1.3 gives a few examples of where neural networks are currently being developed and applied. These examples come from a wide range of areas. Section 1.4 introduces the basics of how a neural network is defined. The key characteristics are the net's architecture and training algorithm. A summary of the notation we shall use and illustrations of some common activation functions are also presented. Section 1.5 provides a brief history of the development of neural networks. Finally, as a transition from the historical context to descriptions of the most fundamental and common neural networks that are the subject of the remaining chapters, we describe the McCulloch-Pitts neuron.

1.2 WHAT IS A NEURAL NET?

1.2.1 Artificial Neural Networks

An *artificial neural network* is an information-processing system that has certain performance characteristics in common with biological neural networks. Artificial neural networks have been developed as generalizations of mathematical models of human cognition or neural biology, based on the assumptions that:

1. Information processing occurs at many simple elements called neurons.
2. Signals are passed between neurons over connection links.
3. Each connection link has an associated weight, which, in a typical neural net, multiplies the signal transmitted.
4. Each neuron applies an activation function (usually nonlinear) to its net input (sum of weighted input signals) to determine its output signal.

A neural network is characterized by (1) its pattern of connections between the neurons (called its *architecture*), (2) its method of determining the weights on the connections (called its *training*, or *learning, algorithm*), and (3) its *activation function*.

Since *what* distinguishes (artificial) neural networks from other approaches to information processing provides an introduction to both *how* and *when* to use neural networks, let us consider the defining characteristics of neural networks further.

A neural net consists of a large number of simple processing elements called *neurons, units, cells,* or *nodes*. Each neuron is connected to other neurons by means of directed communication links, each with an associated weight. The weights represent information being used by the net to solve a problem. Neural nets can be applied to a wide variety of problems, such as storing and recalling data or patterns, classifying patterns, performing general mappings from input patterns to output patterns, grouping similar patterns, or finding solutions to constrained optimization problems.

Each neuron has an internal state, called its *activation* or *activity level,* which is a function of the inputs it has received. Typically, a neuron sends its activation as a signal to several other neurons. It is important to note that a neuron can send only one signal at a time, although that signal is broadcast to several other neurons.

For example, consider a neuron Y, illustrated in Figure 1.1, that receives inputs from neurons X_1, X_2, and X_3. The activations (output signals) of these neurons are x_1, x_2, and x_3, respectively. The weights on the connections from X_1, X_2, and X_3 to neuron Y are w_1, w_2, and w_3, respectively. The net input, y_in, to neuron Y is the sum of the weighted signals from neurons X_1, X_2, and X_3, i.e.,

$$y\_in = w_1x_1 + w_2x_2 + w_3x_3.$$

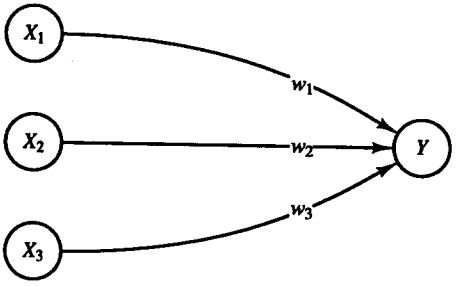

Figure 1.1 A simple (artificial) neuron.

The activation y of neuron Y is given by some function of its net input, $y = f(y\_in)$, e.g., the logistic sigmoid function (an S-shaped curve)

$$f(x) = \frac{1}{1 + \exp(-x)},$$

or any of a number of other activation functions. Several common activation functions are illustrated in Section 1.4.3.

Now suppose further that neuron Y is connected to neurons Z_1 and Z_2, with weights v_1 and v_2, respectively, as shown in Figure 1.2. Neuron Y sends its signal y to each of these units. However, in general, the values received by neurons Z_1 and Z_2 will be different, because each signal is scaled by the appropriate weight, v_1 or v_2. In a typical net, the activations z_1 and z_2 of neurons Z_1 and Z_2 would depend on inputs from several or even many neurons, not just one, as shown in this simple example.

Although the neural network in Figure 1.2 is very simple, the presence of a hidden unit, together with a nonlinear activation function, gives it the ability to solve many more problems than can be solved by a net with only input and output units. On the other hand, it is more difficult to train (i.e., find optimal values for the weights) a net with hidden units. The arrangement of the units (the architecture

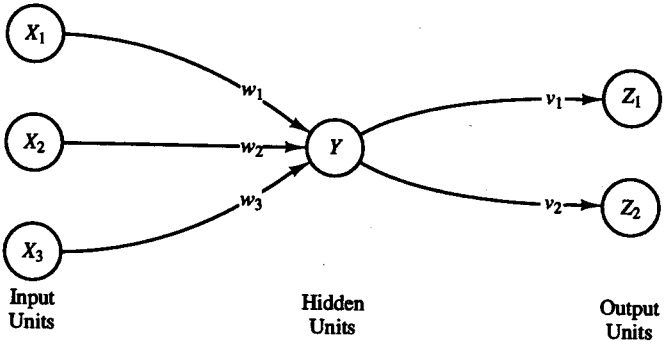

Figure 1.2 A very simple neural network.

of the net) and the method of training the net are discussed further in Section 1.4. A detailed consideration of these ideas for specific nets, together with simple examples of an application of each net, is the focus of the following chapters.

1.2.2 Biological Neural Networks

The extent to which a neural network models a particular biological neural system varies. For some researchers, this is a primary concern; for others, the ability of the net to perform useful tasks (such as approximate or represent a function) is more important than the biological plausibility of the net. Although our interest lies almost exclusively in the computational capabilities of neural networks, we shall present a brief discussion of some features of biological neurons that may help to clarify the most important characteristics of artificial neural networks. In addition to being the original inspiration for artificial nets, biological neural systems suggest features that have distinct computational advantages.

There is a close analogy between the structure of a biological neuron (i.e., a brain or nerve cell) and the processing element (or artificial neuron) presented in the rest of this book. In fact, the structure of an individual neuron varies much less from species to species than does the organization of the system of which the neuron is an element.

A biological neuron has three types of components that are of particular interest in understanding an artificial neuron: its *dendrites, soma,* and *axon.* The many dendrites receive signals from other neurons. The signals are electric impulses that are transmitted across a synaptic gap by means of a chemical process. The action of the chemical transmitter modifies the incoming signal (typically, by scaling the frequency of the signals that are received) in a manner similar to the action of the weights in an artificial neural network.

The soma, or cell body, sums the incoming signals. When sufficient input is received, the cell fires; that is, it transmits a signal over its axon to other cells. It is often supposed that a cell either fires or doesn't at any instant of time, so that transmitted signals can be treated as binary. However, the frequency of firing varies and can be viewed as a signal of either greater or lesser magnitude. This corresponds to looking at discrete time steps and summing all activity (signals received or signals sent) at a particular point in time.

The transmission of the signal from a particular neuron is accomplished by an action potential resulting from differential concentrations of ions on either side of the neuron's axon sheath (the brain's "white matter"). The ions most directly involved are potassium, sodium, and chloride.

A generic biological neuron is illustrated in Figure 1.3, together with axons from two other neurons (from which the illustrated neuron could receive signals) and dendrites for two other neurons (to which the original neuron would send signals). Several key features of the processing elements of artificial neural networks are suggested by the properties of biological neurons, viz., that:

1. The processing element receives many signals.
2. Signals may be modified by a weight at the receiving synapse.
3. The processing element sums the weighted inputs.
4. Under appropriate circumstances (sufficient input), the neuron transmits a single output.
5. The output from a particular neuron may go to many other neurons (the axon branches).

Other features of artificial neural networks that are suggested by biological neurons are:

6. Information processing is local (although other means of transmission, such as the action of hormones, may suggest means of overall process control).
7. Memory is distributed:
   a. Long-term memory resides in the neurons' synapses or weights.
   b. Short-term memory corresponds to the signals sent by the neurons.
8. A synapse's strength may be modified by experience.
9. Neurotransmitters for synapses may be excitatory or inhibitory.

Yet another important characteristic that artificial neural networks share with biological neural systems is *fault tolerance*. Biological neural systems are fault tolerant in two respects. First, we are able to recognize many input signals that are somewhat different from any signal we have seen before. An example of this is our ability to recognize a person in a picture we have not seen before or to recognize a person after a long period of time.

Second, we are able to tolerate damage to the neural system itself. Humans are born with as many as 100 billion neurons. Most of these are in the brain, and most are not replaced when they die [Johnson & Brown, 1988]. In spite of our continuous loss of neurons, we continue to learn. Even in cases of traumatic neural

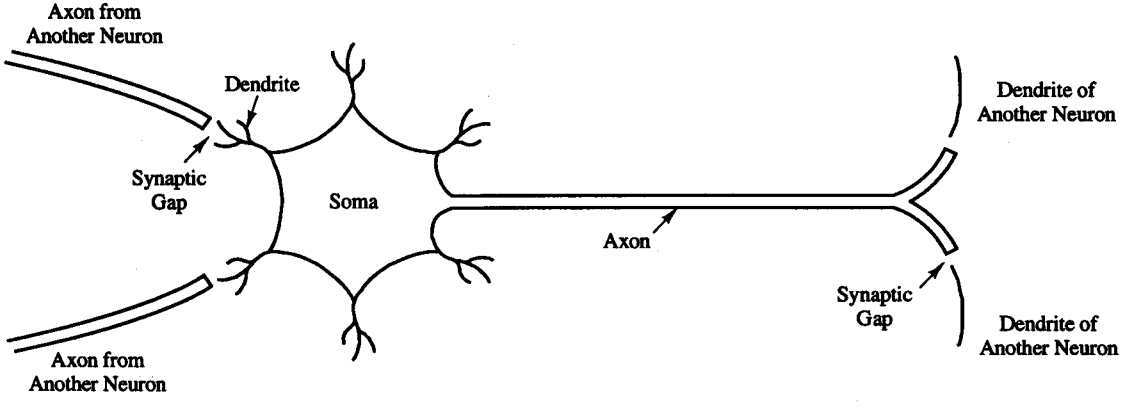

Figure 1.3 Biological neuron.

loss, other neurons can sometimes be trained to take over the functions of the damaged cells. In a similar manner, artificial neural networks can be designed to be insensitive to small damage to the network, and the network can be retrained in cases of significant damage (e.g., loss of data and some connections).

Even for uses of artificial neural networks that are not intended primarily to model biological neural systems, attempts to achieve biological plausibility may lead to improved computational features. One example is the use of a planar array of neurons, as is found in the neurons of the visual cortex, for Kohonen's self-organizing maps (see Chapter 4). The topological nature of these maps has computational advantages, even in applications where the structure of the output units is not itself significant.

Other researchers have found that computationally optimal groupings of artificial neurons correspond to biological bundles of neurons [Rogers & Kabrisky, 1989]. Separating the action of a backpropagation net into smaller pieces to make it more local (and therefore, perhaps more biologically plausible) also allows improvement in computational power (cf. Section 6.2.3) [D. Fausett, 1990].

1.3 WHERE ARE NEURAL NETS BEING USED?

The study of neural networks is an extremely interdisciplinary field, both in its development and in its application. A brief sampling of some of the areas in which neural networks are currently being applied suggests the breadth of their applicability. The examples range from commercial successes to areas of active research that show promise for the future.

1.3.1 Signal Processing

There are many applications of neural networks in the general area of signal processing. One of the first commercial applications was (and still is) to suppress noise on a telephone line. The neural net used for this purpose is a form of ADALINE. (We discuss ADALINES in Chapter 2.) The need for adaptive echo cancelers has become more pressing with the development of transcontinental satellite links for long-distance telephone circuits. The two-way round-trip time delay for the radio transmission is on the order of half a second. The switching involved in conventional echo suppression is very disruptive with path delays of this length. Even in the case of wire-based telephone transmission, the repeater amplifiers introduce echoes in the signal.

The adaptive noise cancellation idea is quite simple. At the end of a long-distance line, the incoming signal is applied to both the telephone system component (called the hybrid) and the adaptive filter (the ADALINE type of neural net). The difference between the output of the hybrid and the output of the ADALINE is the error, which is used to adjust the weights on the ADALINE. The ADALINE is trained to remove the noise (echo) from the hybrid's output signal. (See Widrow and Stearns, 1985, for a more detailed discussion.)

1.3.2 Control

The difficulties involved in backing up a trailer are obvious to anyone who has either attempted or watched a novice attempt this maneuver. However, a driver with experience accomplishes the feat with remarkable ease. As an example of the application of neural networks to control problems, consider the task of training a neural "truck backer-upper" to provide steering directions to a trailer truck attempting to back up to a loading dock [Nguyen & Widrow, 1989; Miller, Sutton, & Werbos, 1990]. Information is available describing the position of the cab of the truck, the position of the rear of the trailer, the (fixed) position of the loading dock, and the angles that the truck and the trailer make with the loading dock. The neural net is able to learn how to steer the truck in order for the trailer to reach the dock, starting with the truck and trailer in any initial configuration that allows enough clearance for a solution to be possible. To make the problem more challenging, the truck is allowed only to back up.

The neural net solution to this problem uses two modules. The first (called the *emulator*) learns to compute the new position of the truck, given its current position and the steering angle. The truck moves a fixed distance at each time step. This module learns the "feel" of how a trailer truck responds to various steering signals, in much the same way as a driver learns the behavior of such a rig. The emulator has several hidden units and is trained using backpropagation (which is the subject of Chapter 6).

The second module is the *controller*. After the emulator is trained, the controller learns to give the correct series of steering signals to the truck so that the trailer arrives at the dock with its back parallel to the dock. At each time step, the controller gives a steering signal and the emulator determines the new position of the truck and trailer. This process continues until either the trailer reaches the dock or the rig jackknifes. The error is then determined and the weights on the controller are adjusted.

As with a driver, performance improves with practice, and the neural controller learns to provide a series of steering signals that direct the truck and trailer to the dock, regardless of the starting position (as long as a solution is possible). Initially, the truck may be facing toward the dock, may be facing away from the dock, or may be at any angle in between. Similarly, the angle between the truck and the trailer may have an initial value short of that in a jack-knife situation. The training process for the controller is similar to the recurrent backpropagation described in Chapter 7.

1.3.3 Pattern Recognition

Many interesting problems fall into the general area of pattern recognition. One specific area in which many neural network applications have been developed is the automatic recognition of handwritten characters (digits or letters). The large

variation in sizes, positions, and styles of writing make this a difficult problem for traditional techniques. It is a good example, however, of the type of information processing that humans can perform relatively easily.

General-purpose multilayer neural nets, such as the backpropagation net (a multilayer net trained by backpropagation) described in Chapter 6, have been used for recognizing handwritten zip codes [Le Cun et al., 1990]. Even when an application is based on a standard training algorithm, it is quite common to customize the architecture to improve the performance of the application. This backpropagation net has several hidden layers, but the pattern of connections from one layer to the next is quite localized.

An alternative approach to the problem of recognizing handwritten characters is the "neocognitron" described in Chapter 7. This net has several layers, each with a highly structured pattern of connections from the previous layer and to the subsequent layer. However, its training is a layer-by-layer process, specialized for just such an application.

1.3.4 Medicine

One of many examples of the application of neural networks to medicine was developed by Anderson et al. in the mid-1980s [Anderson, 1986; Anderson, Golden, and Murphy, 1986]. It has been called the "Instant Physician" [Hecht-Nielsen, 1990]. The idea behind this application is to train an autoassociative memory neural network (the "Brain-State-in-a-Box," described in Section 3.4.2) to store a large number of medical records, each of which includes information on symptoms, diagnosis, and treatment for a particular case. After training, the net can be presented with input consisting of a set of symptoms; it will then find the full stored pattern that represents the "best" diagnosis and treatment.

The net performs surprisingly well, given its simple structure. When a particular set of symptoms occurs frequently in the training set, together with a unique diagnosis and treatment, the net will usually give the same diagnosis and treatment. In cases where there are ambiguities in the training data, the net will give the most common diagnosis and treatment. In novel situations, the net will prescribe a treatment corresponding to the symptom(s) it has seen before, regardless of the other symptoms that are present.

1.3.5 Speech Production

Learning to read English text aloud is a difficult task, because the correct phonetic pronunciation of a letter depends on the context in which the letter appears. A traditional approach to the problem would typically involve constructing a set of rules for the standard pronunciation of various groups of letters, together with a look-up table for the exceptions.

One of the most widely known examples of a neural network approach to

the problem of speech production is NETtalk [Sejnowski and Rosenberg, 1986], a multilayer neural net (i.e., a net with hidden units) similar to those described in Chapter 6. In contrast to the need to construct rules and look-up tables for the exceptions, NETtalk's only requirement is a set of examples of the written input, together with the correct pronunciation for it. The written input includes both the letter that is currently being spoken and three letters before and after it (to provide a context). Additional symbols are used to indicate the end of a word or punctuation. The net is trained using the 1,000 most common English words. After training, the net can read new words with very few errors; the errors that it does make are slight mispronunciations, and the intelligibility of the speech is quite good.

It is interesting that there are several fairly distinct stages to the response of the net as training progresses. The net learns quite quickly to distinguish vowels from consonants; however, it uses the same vowel for all vowels and the same consonant for all consonants at this first stage. The result is a babbling sound. The second stage of learning corresponds to the net recognizing the boundaries between words; this produces a pseudoword type of response. After as few as 10 passes through the training data, the text is intelligible. Thus, the response of the net as training progresses is similar to the development of speech in small children.

1.3.6 Speech Recognition

Progress is being made in the difficult area of speaker-independent recognition of speech. A number of useful systems now have a limited vocabulary or grammar or require retraining for different speakers. Several types of neural networks have been used for speech recognition, including multilayer nets (see Chapter 6) or multilayer nets with recurrent connections (see Section 7.2). Lippmann (1989) summarizes the characteristics of many of these nets.

One net that is of particular interest, both because of its level of development toward a practical system and because of its design, was developed by Kohonen using the self-organizing map (Chapter 4). He calls his net a "phonetic typewriter." The output units for a self-organizing map are arranged in a two-dimensional array (rectangular or hexagonal). The input to the net is based on short segments (a few milliseconds long) of the speech waveform. As the net groups similar inputs, the clusters that are formed are positioned so that different examples of the same phoneme occur on output units that are close together in the output array.

After the speech input signals are mapped to the phoneme regions (which has been done without telling the net what a phoneme is), the output units can be connected to the appropriate typewriter key to construct the phonetic typewriter. Because the correspondence between phonemes and written letters is very regular in Finnish (for which the net was developed), the spelling is often correct. See Kohonen (1988) for a more extensive description.

1.3.7 Business

Neural networks are being applied in a number of business settings [Harston, 1990]. We mention only one of many examples here, the mortgage assessment work by Nestor, Inc. [Collins, Ghosh, & Scofield, 1988a, 1988b].

Although it may be thought that the rules which form the basis for mortgage underwriting are well understood, it is difficult to specify completely the process by which experts make decisions in marginal cases. In addition, there is a large financial reward for even a small reduction in the number of mortgages that become delinquent. The basic idea behind the neural network approach to mortgage risk assessment is to use past experience to train the net to provide more consistent and reliable evaluation of mortgage applications.

Using data from several experienced mortgage evaluators, neural nets were trained to screen mortgage applicants for mortgage origination underwriting and mortgage insurance underwriting. The purpose in each of these is to determine whether the applicant should be given a loan. The decisions in the second kind of underwriting are more difficult, because only those applicants assessed as higher risks are processed for mortgage insurance. The training input includes information on the applicant's years of employment, number of dependents, current income, etc., as well as features related to the mortgage itself, such as the loan-to-value ratio, and characteristics of the property, such as its appraised value. The target output from the net is an "accept" or "reject" response.

In both kinds of underwriting, the neural networks achieved a high level of agreement with the human experts. When disagreement did occur, the case was often a marginal one where the experts would also disagree. Using an independent measure of the quality of the mortgages certified, the neural network consistently made better judgments than the experts. In effect, the net learned to form a consensus from the experience of all of the experts whose actions had formed the basis for its training.

A second neural net was trained to evaluate the risk of default on a loan, based on data from a data base consisting of 111,080 applications, 109,072 of which had no history of delinquency. A total of 4,000 training samples were selected from the data base. Although delinquency can result from many causes that are not reflected in the information available on a loan application, the predictions the net was able to make produced a 12% reduction in delinquencies.

1.4 HOW ARE NEURAL NETWORKS USED?

Let us now consider some of the fundamental features of how neural networks operate. Detailed discussions of these ideas for a number of specific nets are presented in the remaining chapters. The building blocks of our examination here are the network architectures and the methods of setting the weights (training).

We also illustrate several typical activation functions and conclude the section with a summary of the notation we shall use throughout the rest of the text.

1.4.1 Typical Architectures

Often, it is convenient to visualize neurons as arranged in layers. Typically, neurons in the same layer behave in the same manner. Key factors in determining the behavior of a neuron are its activation function and the pattern of weighted connections over which it sends and receives signals. Within each layer, neurons usually have the same activation function and the same pattern of connections to other neurons. To be more specific, in many neural networks, the neurons within a layer are either fully interconnected or not interconnected at all. If any neuron in a layer (for instance, the layer of hidden units) is connected to a neuron in another layer (say, the output layer), then each hidden unit is connected to every output neuron.

The arrangement of neurons into layers and the connection patterns within and between layers is called the *net architecture*. Many neural nets have an input layer in which the activation of each unit is equal to an external input signal. The net illustrated in Figure 1.2 consists of input units, output units, and one hidden unit (a unit that is neither an input unit nor an output unit).

Neural nets are often classified as single layer or multilayer. In determining the number of layers, the input units are not counted as a layer, because they perform no computation. Equivalently, the number of layers in the net can be defined to be the number of layers of weighted interconnect links between the slabs of neurons. This view is motivated by the fact that the weights in a net contain extremely important information. The net shown in Figure 1.2 has two layers of weights.

The single-layer and multilayer nets illustrated in Figures 1.4 and 1.5 are examples of *feedforward* nets—nets in which the signals flow from the input units to the output units, in a forward direction. The fully interconnected competitive net in Figure 1.6 is an example of a *recurrent* net, in which there are closed-loop signal paths from a unit back to itself.

Single-Layer Net

A single-layer net has one layer of connection weights. Often, the units can be distinguished as input units, which receive signals from the outside world, and output units, from which the response of the net can be read. In the typical single-layer net shown in Figure 1.4, the input units are fully connected to output units but are not connected to other input units, and the output units are not connected to other output units. By contrast, the Hopfield net architecture, shown in Figure 3.7, is an example of a single-layer net in which all units function as both input and output units.

For pattern classification, each output unit corresponds to a particular cat-

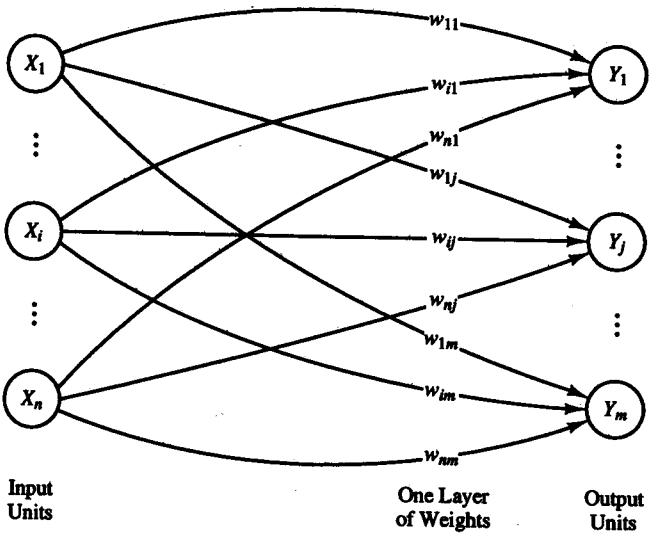

Figure 1.4 A single-layer neural net.

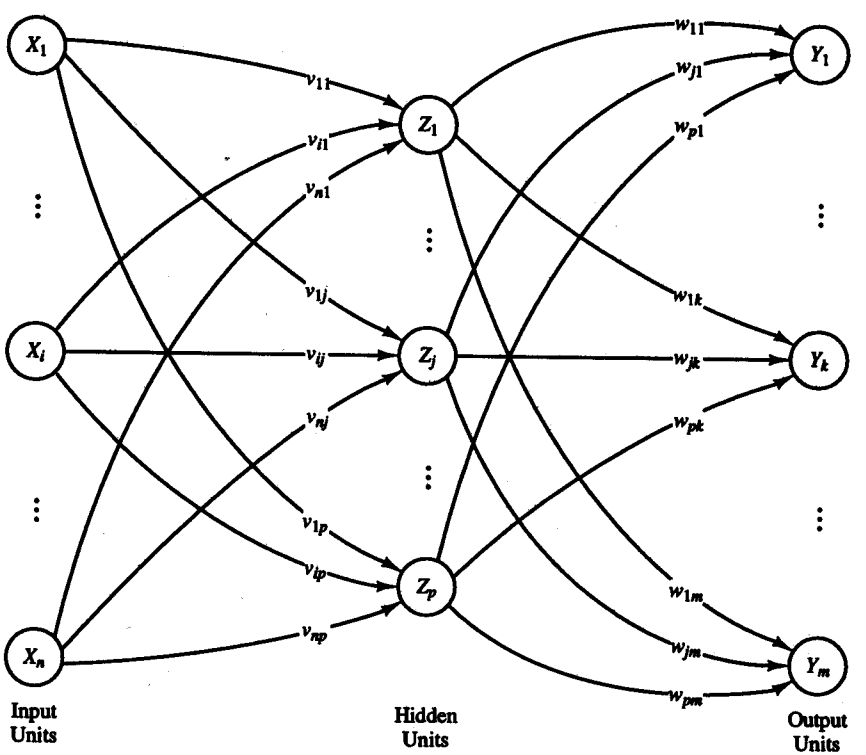

Figure 1.5 A multilayer neural net.

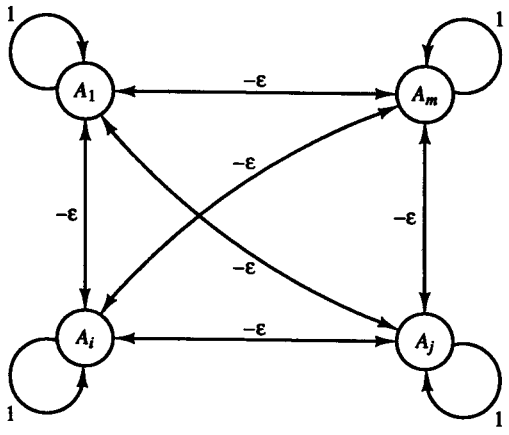

Figure 1.6 Competitive layer.

egory to which an input vector may or may not belong. Note that for a single-layer net, the weights for one output unit do not influence the weights for other output units. For pattern association, the same architecture can be used, but now the overall pattern of output signals gives the response pattern associated with the input signal that caused it to be produced. These two examples illustrate the fact that the same type of net can be used for different problems, depending on the interpretation of the response of the net.

On the other hand, more complicated mapping problems may require a multilayer network. The characteristics of the problems for which a single-layer net is satisfactory are considered in Chapters 2 and 3. The problems that require multilayer nets may still represent a classification or association of patterns; the type of problem influences the choice of architecture, but does not uniquely determine it.

Multilayer net

A multilayer net is a net with one or more layers (or levels) of nodes (the so-called hidden units) between the input units and the output units. Typically, there is a layer of weights between two adjacent levels of units (input, hidden, or output). Multilayer nets can solve more complicated problems than can single-layer nets, but training may be more difficult. However, in some cases, training may be more successful, because it is possible to solve a problem that a single-layer net cannot be trained to perform correctly at all.

Competitive layer

A competitive layer forms a part of a large number of neural networks. Several examples of these nets are discussed in Chapters 4 and 5. Typically, the interconnections between neurons in the competitive layer are not shown in the architecture diagrams for such nets. An example of the architecture for a competitive

layer is given in Figure 1.6; the competitive interconnections have weights of $-\epsilon$. The operation of a winner-take-all competition, MAXNET [Lippman, 1987], is described in Section 4.1.1.

1.4.2 Setting the Weights

In addition to the architecture, the method of setting the values of the weights (training) is an important distinguishing characteristic of different neural nets. For convenience, we shall distinguish two types of training—supervised and unsupervised—for a neural network; in addition, there are nets whose weights are fixed without an iterative training process.

Many of the tasks that neural nets can be trained to perform fall into the areas of mapping, clustering, and constrained optimization. Pattern classification and pattern association may be considered special forms of the more general problem of mapping input vectors or patterns to the specified output vectors or patterns.

There is some ambiguity in the labeling of training methods as supervised or unsupervised, and some authors find a third category, self-supervised training, useful. However, in general, there is a useful correspondence between the type of training that is appropriate and the type of problem we wish to solve. We summarize here the basic characteristics of supervised and unsupervised training and the types of problems for which each, as well as the fixed-weight nets, is typically used.

Supervised training

In perhaps the most typical neural net setting, training is accomplished by presenting a sequence of training vectors, or patterns, each with an associated target output vector. The weights are then adjusted according to a learning algorithm. This process is known as *supervised training*.

Some of the simplest (and historically earliest) neural nets are designed to perform pattern classification, i.e., to classify an input vector as either belonging or not belonging to a given category. In this type of neural net, the output is a bivalent element, say, either 1 (if the input vector belongs to the category) or -1 (if it does not belong). In the next chapter, we consider several simple single-layer nets that were designed or typically used for pattern classification. These nets are trained using a supervised algorithm. The characteristics of a classification problem that determines whether a single-layer net is adequate are considered in Chapter 2 also. For more difficult classification problems, a multilayer net, such as that trained by backpropagation (presented in Chapter 6) may be better.

Pattern association is another special form of a mapping problem, one in which the desired output is not just a "yes" or "no," but rather a pattern. A neural net that is trained to associate a set of input vectors with a corresponding

set of output vectors is called an *associative memory*. If the desired output vector is the same as the input vector, the net is an *autoassociative memory*; if the output target vector is different from the input vector, the net is a *heteroassociative memory*. After training, an associative memory can recall a stored pattern when it is given an input vector that is sufficiently similar to a vector it has learned. Associative memory neural nets, both feedforward and recurrent, are discussed in Chapter 3.

Multilayer neural nets can be trained to perform a nonlinear mapping from an n-dimensional space of input vectors (n-tuples) to an m-dimensional output space—i.e., the output vectors are m-tuples.

The single-layer nets in Chapter 2 (pattern classification nets) and Chapter 3 (pattern association nets) use supervised training (the Hebb rule or the delta rule). Backpropagation (the generalized delta rule) is used to train the multilayer nets in Chapter 6. Other forms of supervised learning are used for some of the nets in Chapter 4 (learning vector quantization and counterpropagation) and Chapter 7. Each learning algorithm will be described in detail, along with a description of the net for which it is used.

Unsupervised training

Self-organizing neural nets group similar input vectors together without the use of training data to specify what a typical member of each group looks like or to which group each vector belongs. A sequence of input vectors is provided, but no target vectors are specified. The net modifies the weights so that the most similar input vectors are assigned to the same output (or cluster) unit. The neural net will produce an exemplar (representative) vector for each cluster formed. Self-organizing nets are described in Chapters 4 (Kohonen self-organizing maps) and Chapter 5 (adaptive resonance theory).

Unsupervised learning is also used for other tasks, in addition to clustering. Examples are included in Chapter 7.

Fixed-weight nets

Still other types of neural nets can solve constrained optimization problems. Such nets may work well for problems that can cause difficulty for traditional techniques, such as problems with conflicting constraints (i.e., not all constraints can be satisfied simultaneously). Often, in such cases, a nearly optimal solution (which the net can find) is satisfactory. When these nets are designed, the weights are set to represent the constraints and the quantity to be maximized or minimized. The Boltzmann machine (without learning) and the continuous Hopfield net (Chapter 7) can be used for constrained optimization problems.

Fixed weights are also used in contrast-enhancing nets (see Section 4.1).

1.4.3 Common Activation Functions

As mentioned before, the basic operation of an artificial neuron involves summing its weighted input signal and applying an output, or activation, function. For the input units, this function is the identity function (see Figure 1.7). Typically, the same activation function is used for all neurons in any particular layer of a neural net, although this is not required. In most cases, a nonlinear activation function is used. In order to achieve the advantages of multilayer nets, compared with the limited capabilities of single-layer nets, nonlinear functions are required (since the results of feeding a signal through two or more layers of linear processing elements—i.e., elements with linear activation functions—are no different from what can be obtained using a single layer).

(*i*) Identity function:

$$f(x) = x \qquad \text{for all } x.$$

Single-layer nets often use a step function to convert the net input, which is a continuously valued variable, to an output unit that is a binary (1 or 0) or bipolar (1 or -1) signal (see Figure 1.8). The use of a *threshold* in this regard is discussed in Section 2.1.2. The binary step function is also known as the threshold function or Heaviside function.

(*ii*) Binary step function (with threshold θ):

$$f(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ 0 & \text{if } x < \theta \end{cases}$$

Sigmoid functions (S-shaped curves) are useful activation functions. The logistic function and the hyperbolic tangent functions are the most common. They are especially advantageous for use in neural nets trained by backpropagation, because the simple relationship between the value of the function at a point and the value of the derivative at that point reduces the computational burden during training.

The logistic function, a sigmoid function with range from 0 to 1, is often

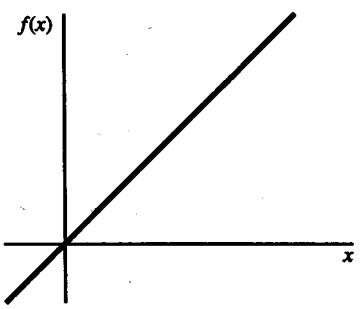

Figure 1.7 Identity function.

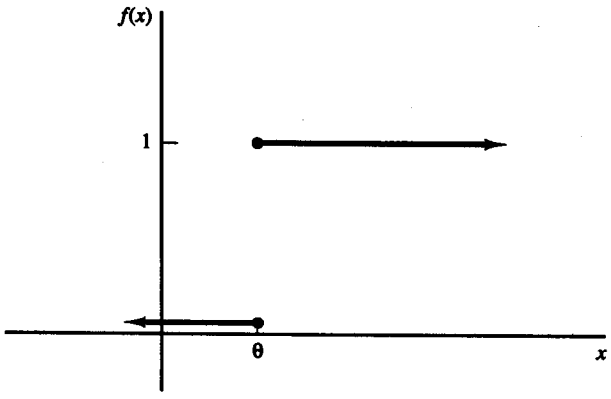

Figure 1.8 Binary step function.

used as the activation function for neural nets in which the desired output values either are binary or are in the interval between 0 and 1. To emphasize the range of the function, we will call it the *binary sigmoid*; it is also called the *logistic sigmoid*. This function is illustrated in Figure 1.9 for two values of the *steepness parameter* σ.

(*iii*) Binary sigmoid:

$$f(x) = \frac{1}{1 + \exp(-\sigma x)}.$$

$$f'(x) = \sigma f(x)\,[1 - f(x)].$$

As is shown in Section 6.2.3, the logistic sigmoid function can be scaled to have any range of values that is appropriate for a given problem. The most common range is from -1 to 1; we call this sigmoid the *bipolar sigmoid*. It is illustrated in Figure 1.10 for $\sigma = 1$.

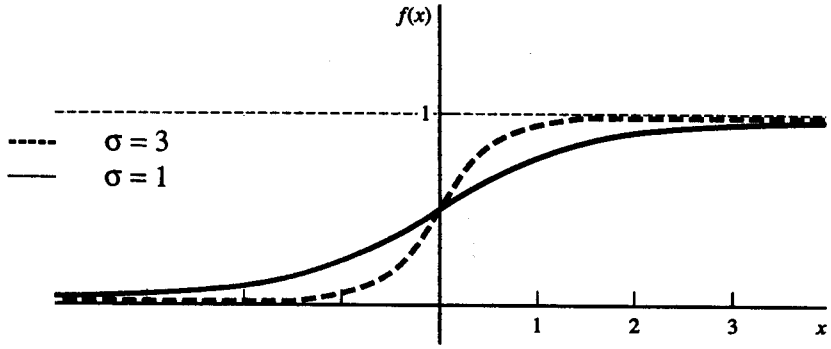

Figure 1.9 Binary sigmoid. Steepness parameters $\sigma = 1$ and $\sigma = 3$.

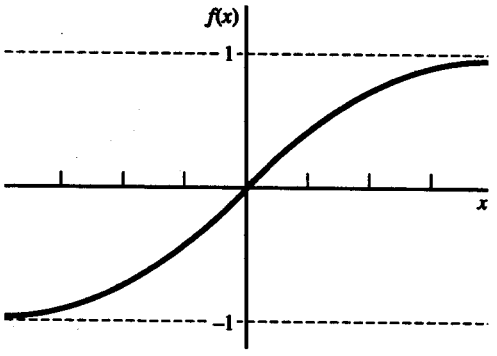

Figure 1.10 Bipolar sigmoid.

(*iv*) Bipolar sigmoid:

$$g(x) = 2f(x) - 1 = \frac{2}{1 + \exp(-\sigma x)} - 1$$

$$= \frac{1 - \exp(-\sigma x)}{1 + \exp(-\sigma x)}.$$

$$g'(x) = \frac{\sigma}{2}[1 + g(x)][1 - g(x)].$$

The bipolar sigmoid is closely related to the hyperbolic tangent function, which is also often used as the activation function when the desired range of output values is between -1 and 1. We illustrate the correspondence between the two for $\sigma = 1$. We have

$$g(x) = \frac{1 - \exp(-x)}{1 + \exp(-x)}.$$

The hyperbolic tangent is

$$h(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

$$= \frac{1 - \exp(-2x)}{1 + \exp(-2x)}.$$

The derivative of the hyperbolic tangent is

$$h'(x) = [1 + h(x)][1 - h(x)].$$

For binary data (rather than continuously valued data in the range from 0 to 1), it is usually preferable to convert to bipolar form and use the bipolar sigmoid or hyperbolic tangent. A more extensive discussion of the choice of activation functions and different forms of sigmoid functions is given in Section 6.2.2.

1.4.4 Summary of Notation

The following notation will be used throughout the discussions of specific neural nets, unless indicated otherwise for a particular net (appropriate values for the parameter depend on the particular neural net model being used and will be discussed further for each model):

x_i, y_j Activations of units X_i, Y_j, respectively:
For input units X_i,

$$x_i = \text{input signal};$$

for other units Y_j,

$$y_j = f(y\_in_j).$$

w_{ij} Weight on connection from unit X_i to unit Y_j:
Beware: Some authors use the opposite convention, with w_{ij} denoting the weight from unit Y_j to unit X_i.

b_j Bias on unit Y_j:
A bias acts like a weight on a connection from a unit with a constant activation of 1 (see Figure 1.11).

y_in_j Net input to unit Y_j:

$$y\_in_j = b_j + \sum_i x_i w_{ij}$$

W Weight matrix:

$$W = \{w_{ij}\}.$$

$\mathbf{w}_{\cdot j}$ Vector of weights:

$$\mathbf{w}_{\cdot j} = (w_{1j}, w_{2j}, \ldots, w_{nj})^T.$$

This is the jth column of the weight matrix.

$\| \mathbf{x} \|$ Norm or magnitude of vector $\mathbf{x}$.

θ_j Threshold for activation of neuron Y_j:
A step activation function sets the activation of a neuron to 1 whenever its net input is greater than the specified threshold value θ_j; otherwise its activation is 0 (see Figure 1.8).

$\mathbf{s}$ Training input vector:

$$\mathbf{s} = (s_1, \ldots, s_i, \ldots, s_n).$$

$\mathbf{t}$ Training (or target) output vector:

$$\mathbf{t} = (t_1, \ldots, t_j, \ldots, t_m).$$

$\mathbf{x}$ Input vector (for the net to classify or respond to):

$$\mathbf{x} = (x_1, \ldots, x_i, \ldots, x_n).$$

Δw_{ij} Change in w_{ij}:

$$\Delta w_{ij} = [w_{ij}\,(\text{new}) - w_{ij}\,(\text{old})].$$

α Learning rate:

The learning rate is used to control the amount of weight adjustment at each step of training.

Matrix multiplication method for calculating net input

If the connection weights for a neural net are stored in a matrix $\mathbf{W} = (w_{ij})$, the net input to unit Y_j (with no bias on unit j) is simply the dot product of the vectors $\mathbf{x} = (x_1, \ldots, x_i, \ldots, x_n)$ and $\mathbf{w}_{\cdot j}$ (the jth column of the weight matrix):

$$y\_in_j = \mathbf{x} \cdot \mathbf{w}_{\cdot j}$$

$$= \sum_{i=1}^{n} x_i w_{ij}\,.$$

Bias

A bias can be included by adding a component $x_0 = 1$ to the vector $\mathbf{x}$, i.e., $\mathbf{x} = (1, x_1, \ldots, x_i, \ldots, x_n)$. The bias is treated exactly like any other weight, i.e., $w_{0j} = b_j$. The net input to unit Y_j is given by

$$y\_in_j = \mathbf{x} \cdot \mathbf{w}_{\cdot j}$$

$$= \sum_{i=0}^{n} x_i w_{ij}$$

$$= w_{0j} + \sum_{i=1}^{n} x_i w_{ij}$$

$$= b_j + \sum_{i=1}^{n} x_i w_{ij}\,.$$

The relation between a bias and a threshold is considered in Section 2.1.2.

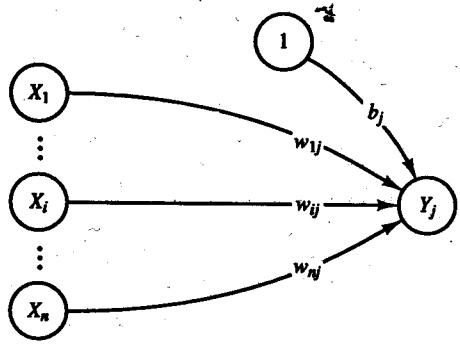

Figure 1.11 Neuron with a bias.

1.5 WHO IS DEVELOPING NEURAL NETWORKS?

This section presents a very brief summary of the history of neural networks, in terms of the development of architectures and algorithms that are widely used today. Results of a primarily biological nature are not included, due to space constraints. They have, however, served as the inspiration for a number of networks that are applicable to problems beyond the original ones studied. The history of neural networks shows the interplay among biological experimentation, modeling, and computer simulation/hardware implementation. Thus, the field is strongly interdisciplinary.

1.5.1 The 1940s: The Beginning of Neural Nets

McCulloch-Pitts neurons

Warren McCulloch and Walter Pitts designed what are generally regarded as the first neural networks [McCulloch & Pitts, 1943]. These researchers recognized that combining many simple neurons into neural systems was the source of increased computational power. The weights on a McCulloch-Pitts neuron are set so that the neuron performs a particular simple logic function, with different neurons performing different functions. The neurons can be arranged into a net to produce any output that can be represented as a combination of logic functions. The flow of information through the net assumes a unit time step for a signal to travel from one neuron to the next. This time delay allows the net to model some physiological processes, such as the perception of hot and cold.

The idea of a threshold such that if the net input to a neuron is greater than the threshold then the unit fires is one feature of a McCulloch-Pitts neuron that is used in many artificial neurons today. However, McCulloch-Pitts neurons are used most widely as logic circuits [Anderson & Rosenfeld, 1988].

McCulloch and Pitts subsequent work [Pitts & McCulloch, 1947] addressed issues that are still important research areas today, such as translation and rotation invariant pattern recognition.

Hebb learning

Donald Hebb, a psychologist at McGill University, designed the first learning law for artificial neural networks [Hebb, 1949]. His premise was that if two neurons were active simultaneously, then the strength of the connection between them should be increased. Refinements were subsequently made to this rather general statement to allow computer simulations [Rochester, Holland, Haibt & Duda, 1956]. The idea is closely related to the correlation matrix learning developed by Kohonen (1972) and Anderson (1972) among others. An expanded form of Hebb learning [McClelland & Rumelhart, 1988] in which units that are simultaneously off also reinforce the weight on the connection between them will be presented in Chapters 2 and 3.

1.5.2 The 1950s and 1960s: The First Golden Age of Neural Networks

Although today neural networks are often viewed as an alternative to (or complement of) traditional computing, it is interesting to note that John von Neumann, the "father of modern computing," was keenly interested in modeling the brain [von Neumann, 1958]. Johnson and Brown (1988) and Anderson and Rosenfeld (1988) discuss the interaction between von Neumann and early neural network researchers such as Warren McCulloch, and present further indication of von Neumann's views of the directions in which computers would develop.

Perceptrons

Together with several other researchers [Block, 1962; Minsky & Papert, 1988 (originally published 1969)], Frank Rosenblatt (1958, 1959, 1962) introduced and developed a large class of artificial neural networks called *perceptrons*. The most typical perceptron consisted of an input layer (the retina) connected by paths with fixed weights to associator neurons; the weights on the connection paths were adjustable. The perceptron learning rule uses an iterative weight adjustment that is more powerful than the Hebb rule. Perceptron learning can be proved to converge to the correct weights if there are weights that will solve the problem at hand (i.e., allow the net to reproduce correctly all of the training input and target output pairs). Rosenblatt's 1962 work describes many types of perceptrons. Like the neurons developed by McCulloch and Pitts and by Hebb, perceptrons use a threshold output function.

The early successes with perceptrons led to enthusiastic claims. However, the mathematical proof of the convergence of iterative learning under suitable assumptions was followed by a demonstration of the limitations regarding what the perceptron type of net can learn [Minsky & Papert, 1969].

Adaline

Bernard Widrow and his student, Marcian (Ted) Hoff [Widrow & Hoff, 1960], developed a learning rule (which usually either bears their names, or is designated the least mean squares or delta rule) that is closely related to the perceptron learning rule. The perceptron rule adjusts the connection weights to a unit whenever the response of the unit is incorrect. (The response indicates a classification of the input pattern.) The delta rule adjusts the weights to reduce the difference between the net input to the output unit and the desired output. This results in the smallest mean squared error. The similarity of models developed in psychology by Rosenblatt to those developed in electrical engineering by Widrow and Hoff is evidence of the interdisciplinary nature of neural networks. The difference in learning rules, although slight, leads to an improved ability of the net to generalize (i.e., respond to input that is similar, but not identical, to that on which it was trained). The Widrow-Hoff learning rule for a single-layer network is a precursor of the backpropagation rule for multilayer nets.

Work by Widrow and his students is sometimes reported as neural network research, sometimes as adaptive linear systems. The name *ADALINE*, interpreted as either ADAptive LInear NEuron or ADAptive LINEar system, is often given to these nets. There have been many interesting applications of ADALINES, from neural networks for adaptive antenna systems [Widrow, Mantey, Griffiths, & Goode, 1967] to rotation-invariant pattern recognition to a variety of control problems, such as broom balancing and backing up a truck [Widrow, 1987; Tolat & Widrow, 1988; Nguyen & Widrow, 1989]. MADALINES are multilayer extensions of ADALINES [Widrow & Hoff, 1960; Widrow & Lehr, 1990].

1.5.3 The 1970s: The Quiet Years

In spite of Minsky and Papert's demonstration of the limitations of perceptrons (i.e., single-layer nets), research on neural networks continued. Many of the current leaders in the field began to publish their work during the 1970s. (Widrow, of course, had started somewhat earlier and is still active.)

Kohonen

The early work of Teuvo Kohonen (1972), of Helsinki University of Technology, dealt with associative memory neural nets. His more recent work [Kohonen, 1982] has been the development of self-organizing feature maps that use a topological structure for the cluster units. These nets have been applied to speech recognition (for Finnish and Japanese words) [Kohonen, Torkkola, Shozakai, Kangas, & Venta, 1987; Kohonen, 1988], the solution of the "Traveling Salesman Problem" [Angeniol, Vaubois, & Le Texier, 1988], and musical composition [Kohonen, 1989b].

Anderson

James Anderson, of Brown University, also started his research in neural networks with associative memory nets [Anderson, 1968, 1972]. He developed these ideas into his "Brain-State-in-a-Box" [Anderson, Silverstein, Ritz, & Jones, 1977], which truncates the linear output of earlier models to prevent the output from becoming too large as the net iterates to find a stable solution (or memory). Among the areas of application for these nets are medical diagnosis and learning multiplication tables. Anderson and Rosenfeld (1988) and Anderson, Pellionisz, and Rosenfeld (1990) are collections of fundamental papers on neural network research. The introductions to each are especially useful.

Grossberg

Stephen Grossberg, together with his many colleagues and coauthors, has had an extremely prolific and productive career. Klimasauskas (1989) lists 146 publications by Grossberg from 1967 to 1988. His work, which is very mathematical and very biological, is widely known [Grossberg, 1976, 1980, 1982, 1987, 1988]. Grossberg is director of the Center for Adaptive Systems at Boston University.

Carpenter

Together with Stephen Grossberg, Gail Carpenter has developed a theory of self-organizing neural networks called *adaptive resonance theory* [Carpenter & Grossberg, 1985, 1987a, 1987b, 1990]. Adaptive resonance theory nets for binary input patterns (ART1) and for continuously valued inputs (ART2) will be examined in Chapter 5.

1.5.4 The 1980s: Renewed Enthusiasm

Backpropagation

Two of the reasons for the "quiet years" of the 1970s were the failure of single-layer perceptrons to be able to solve such simple problems (mappings) as the XOR function and the lack of a general method of training a multilayer net. A method for propagating information about errors at the output units back to the hidden units had been discovered in the previous decade [Werbos, 1974], but had not gained wide publicity. This method was also discovered independently by David Parker (1985) and by LeCun (1986) before it became widely known. It is very similar to yet an earlier algorithm in optimal control theory [Bryson & Ho, 1969]. Parker's work came to the attention of the Parallel Distributed Processing Group led by psychologists David Rumelhart, of the University of California at San Diego, and James McClelland, of Carnegie-Mellon University, who refined and publicized it [Rumelhart, Hinton, & Williams, 1986a, 1986b; McClelland & Rumelhart, 1988].

Hopfield nets

Another key player in the increased visibility of and respect for neural nets is prominent physicist John Hopfield, of the California Institute of Technology. Together with David Tank, a researcher at AT&T, Hopfield has developed a number of neural networks based on fixed weights and adaptive activations [Hopfield, 1982, 1984; Hopfield & Tank, 1985, 1986; Tank & Hopfield, 1987]. These nets can serve as associative memory nets and can be used to solve constraint satisfaction problems such as the "Traveling Salesman Problem." An article in *Scientific American* [Tank & Hopfield, 1987] helped to draw popular attention to neural nets, as did the message of a Nobel prize–winning physicist that, in order to make machines that can do what humans do, we need to study human cognition.

Neocognitron

Kunihiko Fukushima and his colleagues at NHK Laboratories in Tokyo have developed a series of specialized neural nets for character recognition. One example of such a net, called a *neocognitron*, is described in Chapter 7. An earlier self-organizing network, called the cognitron [Fukushima, 1975], failed to recognize position- or rotation-distorted characters. This deficiency was corrected in the neocognitron [Fukushima, 1988; Fukushima, Miyake, & Ito, 1983].

Boltzmann machine

A number of researchers have been involved in the development of nondeterministic neural nets, that is, nets in which weights or activations are changed on the basis of a probability density function [Kirkpatrick, Gelatt, & Vecchi, 1983; Geman & Geman, 1984; Ackley, Hinton, & Sejnowski, 1985; Szu & Hartley, 1987]. These nets incorporate such classical ideas as simulated annealing and Bayesian decision theory.

Hardware implementation

Another reason for renewed interest in neural networks (in addition to solving the problem of how to train a multilayer net) is improved computational capabilities. Optical neural nets [Farhat, Psaltis, Prata, & Paek, 1985] and VLSI implementations [Sivilatti, Mahowald, & Mead, 1987] are being developed.

Carver Mead, of California Institute of Technology, who also studies motion detection, is the coinventor of software to design microchips. He is also cofounder of Synaptics, Inc., a leader in the study of neural circuitry.

Nobel laureate Leon Cooper, of Brown University, introduced one of the first multilayer nets, the *reduced coulomb energy network*. Cooper is chairman of Nestor, the first public neural network company [Johnson & Brown, 1988], and the holder of several patents for information-processing systems [Klimasauskas, 1989].

Robert Hecht-Nielsen and Todd Gutschow developed several digital neurocomputers at TRW, Inc., during 1983–85. Funding was provided by the Defense Advanced Research Projects Agency (DARPA) [Hecht-Nielsen, 1990]. DARPA (1988) is a valuable summary of the state of the art in artificial neural networks (especially with regard to successful applications). To quote from the preface to his book, *Neurocomputing,* Hecht-Nielsen is "an industrialist, an adjunct academic, and a philanthropist without financial portfolio" [Hecht-Nielsen, 1990]. The founder of HNC, Inc., he is also a professor at the University of California, San Diego, and the developer of the counterpropagation network.

1.6 WHEN NEURAL NETS BEGAN: THE McCULLOCH-PITTS NEURON

The McCulloch-Pitts neuron is perhaps the earliest artificial neuron [McCulloch & Pitts, 1943]. It displays several important features found in many neural networks. The requirements for McCulloch-Pitts neurons may be summarized as follows:

1. The activation of a McCulloch-Pitts neuron is binary. That is, at any time step, the neuron either fires (has an activation of 1) or does not fire (has an activation of 0).
2. McCulloch-Pitts neurons are connected by directed, weighted paths.

3. A connection path is excitatory if the weight on the path is positive; otherwise it is inhibitory. All excitatory connections into a particular neuron have the same weights.
4. Each neuron has a fixed threshold such that if the net input to the neuron is greater than the threshold, the neuron fires.
5. The threshold is set so that inhibition is absolute. That is, any nonzero inhibitory input will prevent the neuron from firing.
6. It takes one time step for a signal to pass over one connection link.

The simple example of a McCulloch-Pitts neuron shown in Figure 1.12 illustrates several of these requirements. The connection from X_1 to Y is excitatory, as is the connection from X_2 to Y. These excitatory connections have the same (positive) weight because they are going into the same unit.

The threshold for unit Y is 4; for the values of the excitatory and inhibitory weights shown, this is the only integer value of the threshold that will allow Y to fire sometimes, but will prevent it from firing if it receives a nonzero signal over the inhibitory connection.

It takes one time step for the signals to pass from the X units to Y; the activation of Y at time t is determined by the activations of X_1, X_2, and X_3 at the previous time, $t - 1$. The use of discrete time steps enables a network of McCulloch-Pitts neurons to model physiological phenomena in which there is a time delay; such an example is given in Section 1.6.3.

1.6.1 Architecture

In general, a McCulloch-Pitts neuron Y may receive signals from any number of other neurons. Each connection path is either excitatory, with weight $w > 0$, or inhibitory, with weight $-p$ ($p > 0$). For convenience, in Figure 1.13, we assume there are n units, $X_1, \ldots, X_n$, which send excitatory signals to unit Y, and m units, $X_{n+1}, \ldots, X_{n+m}$, which send inhibitory signals. The activation function for unit Y is

$$f(y\_in) = \begin{cases} 1 & \text{if } y\_in \geq \theta \\ 0 & \text{if } y\_in < \theta \end{cases}$$

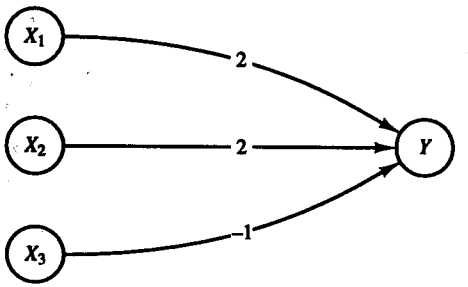

Figure 1.12 A simple McCulloch-Pitts neuron Y.

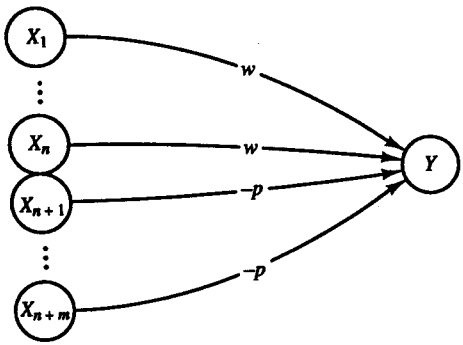

Figure 1.13 Architecture of a McCulloch-Pitts neuron Y.

where y_in is the total input signal received and θ is the threshold. The condition that inhibition is absolute requires that θ for the activation function satisfy the inequality

$$\theta > nw - p.$$

Y will fire if it receives k or more excitatory inputs and no inhibitory inputs, where

$$kw \geq \theta > (k - 1)w.$$

Although all excitatory weights coming into any particular unit must be the same, the weights coming into one unit, say, Y_1, do not have to be the same as the weights coming into another unit, say Y_2.

1.6.2 Algorithm

The weights for a McCulloch-Pitts neuron are set, together with the threshold for the neuron's activation function, so that the neuron will perform a simple logic function. Since analysis, rather than a training algorithm, is used to determine the values of the weights and threshold, several examples of simple McCulloch-Pitts neurons are presented in this section. Using these simple neurons as building blocks, we can model any function or phenomenon that can be represented as a logic function. In Section 1.6.3, an example is given of how several of these simple neurons can be combined to model an interesting physiological phenomenon.

Simple networks of McCulloch-Pitts neurons, each with a threshold of 2, are shown in Figures 1.14–1.17. The activation of unit X_i at time t is denoted $x_i(t)$. The activation of a neuron X_i at time t is determined by the activations, at time $t - 1$, of the neurons from which it receives signals.

Logic functions will be used as simple examples for a number of neural nets. The binary form of the functions for AND, OR, and AND NOT are defined here for reference. Each of these functions acts on two input values, denoted x_1 and x_2, and produces a single output value y.

AND

The AND function gives the response "true" if both input values are "true"; otherwise the response is "false." If we represent "true" by the value 1, and "false" by 0, this gives the following four training input, target output pairs:

x_1	x_2	→	y
1	1		1
1	0		0
0	1		0
0	0		0

Example 1.1 A McCulloch-Pitts Neuron for the AND Function

The network in Figure 1.14 performs the mapping of the logical AND function. The threshold on unit Y is 2.

OR

The OR function gives the response "true" if either of the input values is "true"; otherwise the response is "false." This is the "inclusive or," since both input values may be "true" and the response is still "true." Representing "true" as 1, and "false" as 0, we have the following four training input, target output pairs:

x_1	x_2	→	y
1	1		1
1	0		1
0	1		1
0	0		0

Example 1.2 A McCulloch-Pitts Neuron for the OR Function

The network in Figure 1.15 performs the logical OR function. The threshold on unit Y is 2.

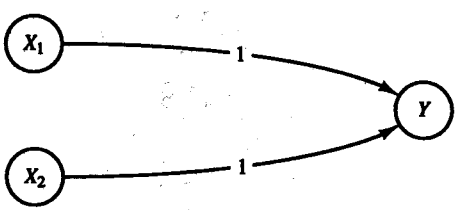

Figure 1.14 A McCulloch-Pitts neuron to perform the logical AND function.

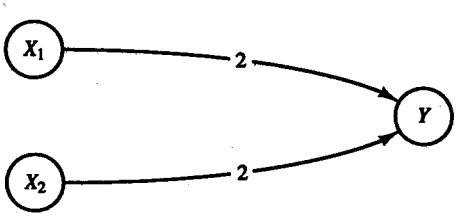

Figure 1.15 A McCulloch-Pitts neuron to perform the logical OR function.

AND NOT

The AND NOT function is an example of a logic function that is not symmetric in its treatment of the two input values. The response is "true" if the first input value, x_1, is "true" and the second input value, x_2, is "false"; otherwise the response is "false." Using a binary representation of the logical input and response values, the four training input, target output pairs are:

x_1	x_2	→	y
1	1		0
1	0		1
0	1		0
0	0		0

Example 1.3 A McCulloch-Pitts Neuron for the AND NOT Function

The net in Figure 1.16 performs the function x_1 AND NOT x_2. In other words, neuron Y fires at time t if and only if unit X_1 fires at time $t - 1$ and unit X_2 does not fire at time $t - 1$. The threshold for unit Y is 2.

1.6.3 Applications

XOR

The XOR (exclusive or) function gives the response "true" if exactly one of the input values is "true"; otherwise the response is "false." Using a binary representation, the four training input, target output pairs are:

x_1	x_2	→	y
1	1		0
1	0		1
0	1		1
0	0		0

Example 1.4 A McCulloch-Pitts Net for the XOR Function

The network in Figure 1.17 performs the logical XOR function. XOR can be expressed as

$$x_1 \text{ XOR } x_2 \leftrightarrow (x_1 \text{ AND NOT } x_2) \text{ OR } (x_2 \text{ AND NOT } x_1).$$

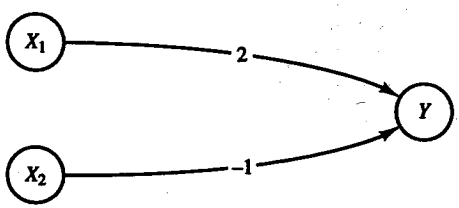

Figure 1.16 A McCulloch-Pitts neuron to perform the logical AND NOT function.

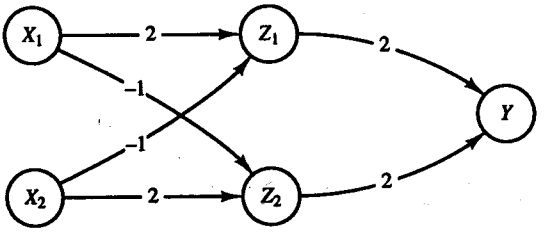

Figure 1.17 A McCulloch-Pitts neural net to perform the logical XOR function.

Thus, $y = x_1$ XOR x_2 is found by a two-layer net. The first layer forms

$$z_1 = x_1 \text{ AND NOT } x_2$$

and

$$z_2 = x_2 \text{ AND NOT } x_1.$$

The second layer consists of

$$y = z_1 \text{ OR } z_2.$$

Units Z_1, Z_2, and Y each have a threshold of 2.

Hot and cold

Example 1.5 Modeling the Perception of Hot and Cold with a McCulloch-Pitts Net

It is a well-known and interesting physiological phenomenon that if a cold stimulus is applied to a person's skin for a very short period of time, the person will perceive heat. However, if the same stimulus is applied for a longer period, the person will perceive cold. The use of discrete time steps enables the network of McCulloch-Pitts neurons shown in Figure 1.18 to model this phenomenon. The example is an elaboration of one originally presented by McCulloch and Pitts [1943]. The model is designed to give only the first perception of heat or cold that is received by the perceptor units.

In the figure, neurons X_1 and X_2 represent receptors for heat and cold, respectively, and neurons Y_1 and Y_2 are the counterpart perceptors. Neurons Z_1

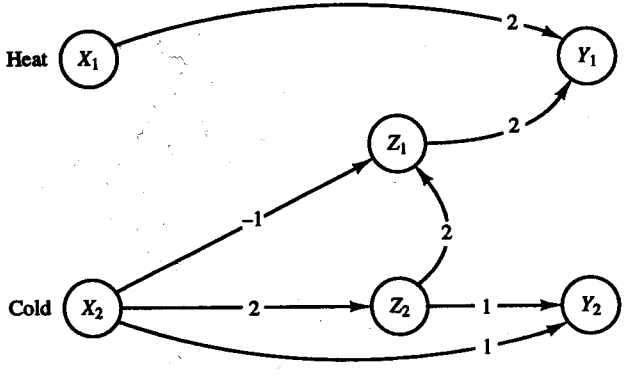

Figure 1.18 A network of McCulloch-Pitts neurons to model the perception of heat and cold.

and Z_2 are auxiliary units needed for the problem. Each neuron has a threshold of 2, i.e., it fires (sets its activation to 1) if the net input it receives is ≥ 2. Input to the system will be (1,0) if heat is applied and (0,1) if cold is applied. The desired response of the system is that cold is perceived if a cold stimulus is applied for two time steps, i.e.,

$$y_2(t) = x_2(t - 2) \text{ AND } x_2(t - 1).$$

The activation of unit Y_2 at time t is $y_2(t)$; $y_2(t) = 1$ if cold is perceived, and $y_2(t) = 0$ if cold is not perceived.

In order to model the physical phenomenon described, it is also required that heat be perceived if either a hot stimulus is applied or a cold stimulus is applied briefly (for one time step) and then removed. This condition is expressed as

$$y_1(t) = \{x_1(t - 1)\} \text{ OR } \{x_2(t - 3) \text{ AND NOT } x_2(t - 2)\}.$$

To see that the net shown in Figure 1.18 does in fact represent the two logical statements required, consider first the neurons that determine the response of Y_1 at time t (illustrated in Figure 1.19). The figure shows that

$$y_1(t) = x_1(t - 1) \text{ OR } z_1(t - 1).$$

Now consider the neurons (illustrated in Figure 1.20) that determine the response of unit Z_1 at time $t - 1$. This figure shows that

$$z_1(t - 1) = z_2(t - 2) \text{ AND NOT } x_2(t - 2).$$

Finally, the response of unit Z_2 at time $t - 2$ is simply the value of X_2 at the previous time step:

$$z_2(t - 2) = x_2(t - 3).$$

Substituting in the preceding expression for $y_1(t)$ gives

$$y_1(t) = \{x_1(t - 1)\} \text{ OR } \{x_2(t - 3) \text{ AND NOT } x_2(t - 2)\}.$$

The analysis for the response of neuron Y_2 at time t proceeds in a similar manner. Figure 1.21 shows that $y_2(t) = z_2(t - 1)$ AND $x_2(t - 1)$. However, as

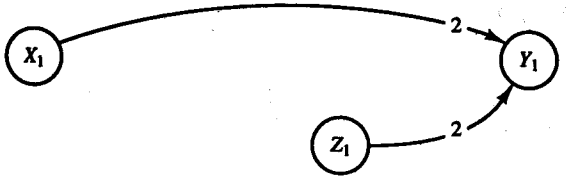

Figure 1.19 The neurons that determine the response of unit Y_1.

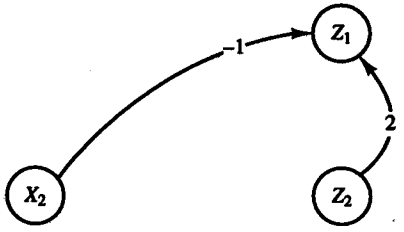

Figure 1.20 The neurons that determine the response of unit Z_1.

before, $z_2(t - 1) = x_2(t - 2)$; substituting in the expression for $y_2(t)$ then gives

$$y_2(t) = x_2(t - 2) \text{ AND } x_2(t - 1),$$

as required.

It is also informative to trace the flow of activations through the net, starting with the presentation of a stimulus at $t = 0$. Case 1, a cold stimulus applied for one time step and then removed, is illustrated in Figure 1.22. Case 2, a cold stimulus applied for two time steps, is illustrated in Figure 1.23, and case 3, a hot stimulus applied for one time step, is illustrated in Figure 1.24. In each case, only the activations that are known at a particular time step are indicated. The weights on the connections are as in Figure 1.18.

Case 1: A cold stimulus applied for one time step.

The activations that are known at $t = 0$ are shown in Figure 1.22(a).

The activations that are known at $t = 1$ are shown in Figure 1.22(b). The activations of the input units are both 0, signifying that the cold stimulus presented at $t = 0$ was removed after one time step. The activations of Z_1 and Z_2 are based on the activations of X_2 at $t = 0$.

The activations that are known at $t = 2$ are shown in Figure 1.22(c). Note that the activations of the input units are not specified, since their value at $t = 2$ does not determine the first response of the net to the situation being modeled. Although the responses of the perceptor units are determined, no perception of hot or cold has reached them yet.

The activations that are known at $t = 3$ are shown in Figure 1.22(d). A perception of heat is indicated by the fact that unit Y_1 has an activation of 1 and unit Y_2 has an activation of 0.

Case 2: A cold stimulus applied for two time steps.

The activations that are known at $t = 0$ are shown in Figure 1.23(a), and those that are known at $t = 1$ are shown in Figure 1.23(b).

The activations that are known at $t = 2$ are shown in Figure 1.23(c). Note that the activations of the input units are not specified, since the first response of the net to the cold stimulus being applied for two time steps is not influenced by whether or not the stimulus is removed after the two steps. Although the responses of the auxiliary units Z_1 and Z_2 are indicated, the responses of the perceptor units are determined by the activations of all of the units at $t = 1$.

Case 3: A hot stimulus applied for one time step.

The activations that are known at $t = 0$ are shown in Figure 1.24(a).

The activations that are known at $t = 1$ are shown in Figure 1.24(b). Unit Y_1 fires because it has received a signal from X_1. Y_2 does not fire because it requires input signals from both X_2 and Z_2 in order to fire, and X_2 had an activation of 0 at $t = 0$.

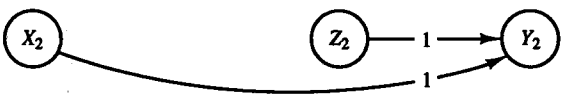

Figure 1.21 The neurons that determine the response of unit Y_2.

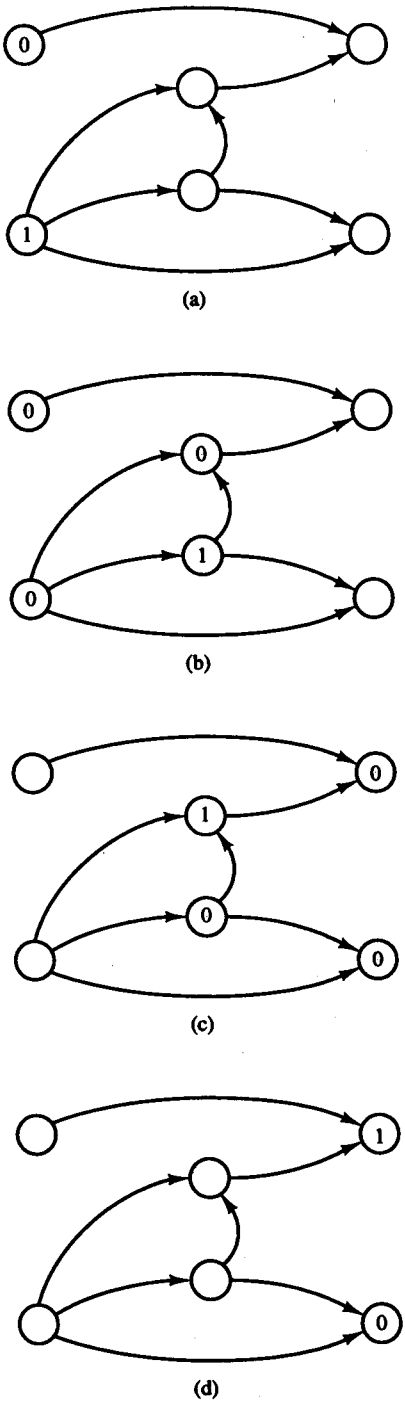

Figure 1.22 A cold stimulus applied for one time step.
Activations at (a) $t = 0$, (b) $t = 1$, (c) $t = 2$, and (d) $t = 3$.

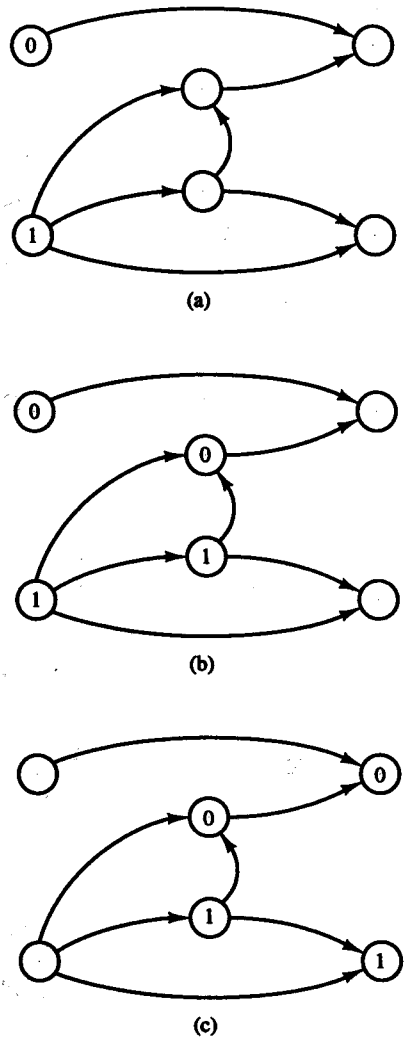

Figure 1.23 A cold stimulus applied for two time steps.
Activations at (a) $t = 0$, (b) $t = 1$, and (c) $t = 2$.

1.7 SUGGESTIONS FOR FURTHER STUDY

1.7.1 Readings

Many of the applications and historical developments we have summarized in this chapter are described in more detail in two collections of original research:

- *Neurocomputing: Foundations of Research* [Anderson & Rosenfeld, 1988].
- *Neurocomputing 2: Directions for Research* [Anderson, Pellionisz & Rosenfeld, 1990].

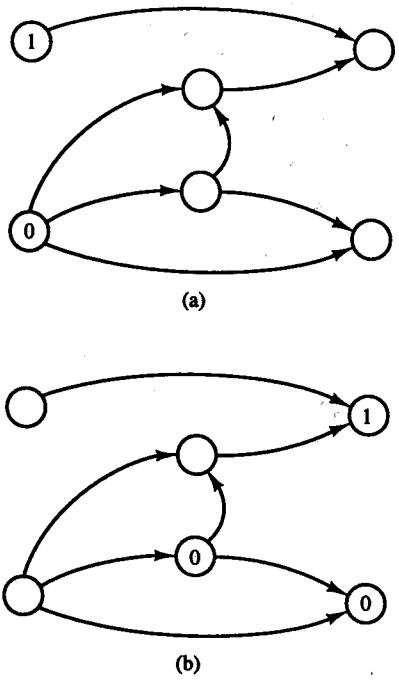

Figure 1.24 A hot stimulus applied for one time step.
Activations at (a) $t = 0$ and (b) $t = 1$.

These contain useful papers, along with concise and insightful introductions explaining the significance and key results of each paper.

The *DARPA Neural Network Study* (1988) also provides descriptions of both the theoretical and practical state of the art of neural networks that year.

Nontechnical introductions

Two very readable nontechnical introductions to neural networks, with an emphasis on the historical development and the personalities of the leaders in the field, are:

- *Cognizers* [Johnson & Brown, 1988].
- *Apprentices of Wonder: Inside the Neural Network Revolution* [Allman, 1989].

Applications

Among the books dealing with neural networks for particular types of applications are:

- *Neural Networks for Signal Processing* [Kosko, 1992b].
- *Neural Networks for Control* [Miller, Sutton, & Werbos, 1990].

- *Simulated Annealing and Boltzmann Machines* [Aarts & Korst, 1989].
- *Adaptive Pattern Recognition and Neural Networks* [Pao, 1989].
- *Adaptive Signal Processing* [Widrow & Sterns, 1985].

History

The history of neural networks is a combination of progress in experimental work with biological neural systems, computer modeling of biological neural systems, the development of mathematical models and their applications to problems in a wide variety of fields, and hardware implementation of these models. In addition to the collections of original papers already mentioned, in which the introductions to each paper provide historical perspectives, *Embodiments of Mind* [McCulloch, 1988] is a wonderful selection of some of McCulloch's essays. *Perceptrons* [Minsky & Papert, 1988] also places the development of neural networks into historical context.

Biological neural networks

Introduction to Neural and Cognitive Modeling [Levine, 1991] provides extensive information on the history of neural networks from a mathematical and psychological perspective. For additional writings from a biological point of view, see *Neuroscience and Connectionist Theory* [Gluck & Rumelhart, 1990] and *Neural and Brain Modeling* [MacGregor, 1987].